



Tutorial 1 - Introduction to Docker

Learning Goals

By the end of this class we will be able to:

- Explain what containers are and why they're useful
- Install and verify Docker on our system
- Run, stop, and inspect containers
- Build our own Docker image from a Python script

0. Warm-up Discussion

- How do you currently run Python projects? (venv, conda, system Python?)
- Have you ever had "it works on my machine but not on ours 🍷"?

💡 Motivation: Docker solves **reproducibility** and **isolation**

1. What is Docker?



Docker is not a virtual machine

- **Virtual Machine vs Container**
 - VM = full OS + app → heavy (GBs, slow)
 - Container = app + dependencies only → lightweight, fast
- Docker uses **images** (blueprints) to create **containers** (running instances)
- Analogy:
 - Image = recipe
 - Container = pizza you baked 🍕

✅ Containers share the host's kernel → smaller and faster.

2. Install Docker (10 min)

- Download **Docker Desktop**: <https://www.docker.com/get-started>
- Open terminal:

```
docker --version
```

✅ Expected: Docker version ...

⚠ Troubleshooting:

- Mac/Linux: may need sudo
- Windows: enable WSL2 backend

3. Run our first container

```
docker run hello-world
```

What is it doing?

- This downloads the image from **Docker Hub**



What is Docker Hub? <https://hub.docker.com>

- Container prints a hello message and exits

Run something interactive:

```
docker run -it ubuntu bash
```

- Try commands

```
ls  
pwd  
echo "Hello from inside Docker"
```



What is happening? Where are we running our commands?

- Exit

```
exit
```

4. Manage containers (useful commands)

- List running containers:

```
docker ps
```

- List all containers (including stopped):

```
docker ps -a
```

- Stop container

```
docker stop <id>
```

- Remove container

```
docker rm <id>
```



What is the difference between stopping and removing a container? Can we first remove and then stop?

- View logs

```
docker logs <id>
```

5. Volumes: persist data (15 min)

What is a Volume?

- By default, everything inside a container is **temporary**
- If you stop/remove the container → all files inside disappear
- **Volumes** are special folders that allow data to **persist outside the container's lifecycle**
- They can be shared between:
 - Container ↔ Host machine
 - Multiple containers

In ML/AI, volumes are crucial for:



- Storing datasets (so they don't vanish when a container stops)
- Saving trained models for reuse
- Sharing logs/results between services

Let's do an experiment

Without a volume

```
docker run -it ubuntu bash
echo "test" > file.txt
exit
```

Restart the same container

```
docker start -ai <id>
ls # file.txt still exists
```

But if we remove the container, the file is gone

```
docker rm <id>
```

With a volume

```
mkdir data
docker run -it -v $(pwd)/data:/data ubuntu bash
echo "persistent!" > /data/file.txt
exit
```

✅ The file is now stored on our **host machine** inside `./data/`.

Even if you delete the container, the file remains.

6. Build our first image

1. Create folder `myapp/` with `app.py` :

```
print("Hello containers!")
```

2. Create `Dockerfile` with the following content:

```
FROM python:3.10
COPY app.py /app.py
CMD ["python", "app.py"]
```

3. Build:

```
docker build -t hello-docker .
```

4. Run:

```
docker run hello-docker
```

🤔 What's happening here?

Dockerfile is like a recipe that tells Docker how to build the image.

Line by line:

- `FROM python:3.10` → Use Python 3.10 as the **base image** (includes Python installed).



Can you find `python:3.10` image in <https://hub.docker.com/>?

- `COPY app.py /app.py` → Copy our script into the container's filesystem.
- `CMD ["python", "app.py"]` → Command that runs when the container starts.

Build step:

- `docker build -t hello-docker .`
 - `-t hello-docker` → tag the image with a name.
 - `.` → build context (current folder).
 - Docker executes the instructions in Dockerfile to create an image.

Run step:

- `docker run hello-docker`
 - Creates a container from the image.
 - Executes the default command (`python app.py`).



Questions

- What is the role of the **Dockerfile** compared to `docker run`?
- Why do we need a **base image** like `python:3.10` ?
- Where exactly does the file `/app.py` live once inside the container?
- How is this different from just running `python app.py` on our host machine?



Wrap-up

- What is the difference between **image** and **container**?

- Why use **volumes**?